ФГБОУ ВО «Ивановский государственный энергетический университет имени В.И. Ленина»
Факультет информатики и вычислительной техники Кафедра программного обеспечения компьютерных систем

ОТЧЕТ
по лабораторной работе №5 Тема: «Нейроконтроллер»

Выполнил:
студент гр. 4-42хх Лашманов В.С.

Проверил:
к.т.н., доцент Сидоров С.Г.

Иваново, 2026

Оглавление

Задание 3

Алгоритм 4

Интерфейс программы 7

Код программы 10

Графики 17

Контрольные вопросы 19

Задание

Разработать собственную реализацию нейроконтроллера, предназначенного для анализа рыночных показателей и выбора инвестиционного решения (покупка, удержание или продажа) на основе заданных входных параметров.

Алгоритм

Разрабатываемый нейроконтроллер предназначен для выработки решений на основе ограниченного набора обучающих примеров. После завершения обучения система способна формировать рекомендации и для новых рыночных ситуаций, которые ранее не встречались.

Входными данными для нейронной сети являются характеристики текущего состояния рынка. В данной работе используются следующие параметры:

- **RSI** — индекс относительной силы, отражающий степень перекупленности или перепроданности актива;
- **MACD** — показатель силы и направления тренда;
- **Volume** — объём торгов;
- **Volatility** — уровень рыночной волатильности;
- **Trend** — направление движения рынка (нисходящий, нейтральный или восходящий).

Каждый из этих параметров подаётся на вход сети и участвует в формировании итогового решения.

На выходе нейросеть должна выбрать одно из трёх действий:

- покупка актива (BUY);
- удержание позиции (HOLD);
- продажа актива (SELL).

Обучающая выборка сформирована таким образом, чтобы отражать различные рыночные ситуации. Например, при устойчивом росте и положительных значениях индикаторов сеть должна склоняться к покупке. При выраженном падении — к продаже. В более нейтральных условиях предпочтительным является удержание позиции. Также учитываются особые случаи, такие как перекупленность или перепроданность, при которых поведение может отличаться от стандартных сценариев.

Работа программы разделена на несколько этапов.

Сначала выполняется инициализация: веса всех связей между нейронами задаются случайными значениями из небольшого диапазона (примерно от –0.5 до 0.5). Это необходимо для того, чтобы сеть могла начать обучение без заранее заданных предпочтений.

Далее следует этап обучения. На каждом шаге выбирается очередной элемент обучающей выборки (при достижении конца происходит возврат к началу). Перед подачей в сеть входные данные нормализуются, чтобы привести их к сопоставимому диапазону значений.

Затем выполняется прямое распространение сигнала. Сначала вычисляются значения

скрытого слоя: для каждого нейрона определяется сумма произведений входных сигналов на соответствующие веса, после чего к результату применяется сигмоидальная функция активации. Аналогично рассчитываются значения выходного слоя.

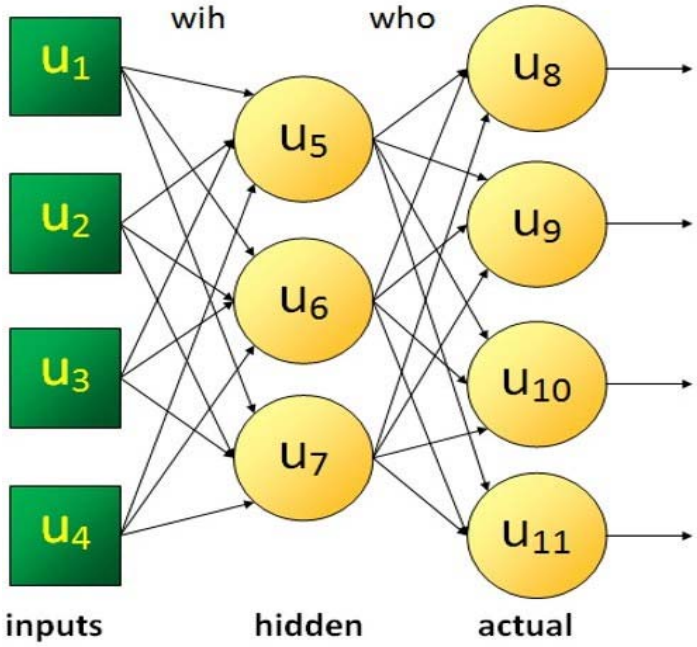

Рисунок 1 – Архитектура нейроконтроллера

После получения выходных значений вычисляется ошибка текущего шага обучения. Она определяется как сумма квадратов разностей между ожидаемыми и фактическими значениями.

Далее выполняется обратное распространение ошибки. Сначала определяется ошибка для выходных нейронов — как разность между целевым и полученным значением, умноженная на производную функции активации. Затем аналогичным образом рассчитывается ошибка скрытого слоя с учётом вкладов всех выходных нейронов.

После этого производится корректировка весов. Каждый вес изменяется пропорционально величине ошибки, значению сигнала и коэффициенту обучения. Таким образом, сеть постепенно подстраивает свои параметры, стремясь уменьшить общую ошибку.

После завершения этапа обучения выполняется проверка сети. Для каждого элемента обучающей выборки определяется, какое действие выбирает нейросеть. Итоговый результат сравнивается с ожидаемым, после чего вычисляется общее количество правильных ответов и их процент.

Заключительный этап — тестирование. На этом этапе в сеть подаются новые наборы

входных данных, которые не использовались при обучении. Это позволяет оценить способность модели обобщать полученные знания и корректно реагировать на новые рыночные условия.

Таким образом, реализованный алгоритм обеспечивает постепенное обучение нейросети на примерах и формирование устойчивой зависимости между входными рыночными параметрами и рекомендуемыми действиями инвестора.

Интерфейс программы

Программа реализована в виде консольного приложения, и всё взаимодействие с пользователем осуществляется через текстовый вывод в терминал. Интерфейс не содержит графических элементов, однако он организован таким образом, чтобы последовательно отображать все этапы работы нейронной сети и давать полное представление о процессе обучения и тестирования. Скриншоты интерфейса представлены на рисунках 2-3.

```
=== ОБУЧЕНИЕ ИНВЕСТИЦИОННОГО СОВЕТНИКА ===

Шаг 0 | MSE = 0.316123
Шаг 2000 | MSE = 0.216017
Шаг 4000 | MSE = 0.0285196
Шаг 6000 | MSE = 0.272417
Шаг 8000 | MSE = 0.000543184
Шаг 10000 | MSE = 0.044857
Шаг 12000 | MSE = 0.00759574
Шаг 14000 | MSE = 0.0111341
Шаг 16000 | MSE = 0.0213762
Шаг 18000 | MSE = 0.0232932

=== ПРОВЕРКА НА ОБУЧАЮЩЕЙ ВЫБОРКЕ ===

Ситуация 1: RSI=75 MACD=2.5 Vol=0.9 Volat=0.3 Trend=1 | Решение: BUY +
  Вероятности: BUY=0.957922 HOLD=0.100192 SELL=0.0253668

Ситуация 2: RSI=80 MACD=3 Vol=1 Volat=0.4 Trend=1 | Решение: BUY +
  Вероятности: BUY=0.931857 HOLD=0.0864053 SELL=0.0534716

Ситуация 3: RSI=65 MACD=1.2 Vol=0.7 Volat=0.5 Trend=1 | Решение: BUY +
  Вероятности: BUY=0.522412 HOLD=0.432974 SELL=0.0429322

Ситуация 4: RSI=60 MACD=0.8 Vol=0.6 Volat=0.5 Trend=1 | Решение: HOLD +
Ситуация 5: RSI=50 MACD=0.1 Vol=0.5 Volat=0.4 Trend=0 | Решение: HOLD +
Ситуация 6: RSI=45 MACD=-0.2 Vol=0.4 Volat=0.3 Trend=0 | Решение: HOLD +
Ситуация 7: RSI=55 MACD=0.3 Vol=0.6 Volat=0.4 Trend=0 | Решение: HOLD +
Ситуация 8: RSI=35 MACD=-1.5 Vol=0.8 Volat=0.7 Trend=-1 | Решение: SELL +
Ситуация 9: RSI=30 MACD=-2 Vol=0.9 Volat=0.8 Trend=-1 | Решение: SELL +
Ситуация 10: RSI=25 MACD=-3 Vol=1 Volat=0.9 Trend=-1 | Решение: SELL +
Ситуация 11: RSI=20 MACD=-4 Vol=0.9 Volat=1 Trend=-1 | Решение: SELL +
Ситуация 12: RSI=85 MACD=2 Vol=0.8 Volat=0.6 Trend=1 | Решение: HOLD +
Ситуация 13: RSI=90 MACD=1.5 Vol=0.7 Volat=0.7 Trend=0 | Решение: HOLD +
Ситуация 14: RSI=15 MACD=-3.5 Vol=0.6 Volat=0.8 Trend=-1 | Решение: SELL +
Ситуация 15: RSI=10 MACD=-5 Vol=0.5 Volat=1 Trend=-1 | Решение: SELL +
Ситуация 16: RSI=70 MACD=2.8 Vol=1 Volat=0.5 Trend=1 | Решение: BUY +
Ситуация 17: RSI=40 MACD=-1.8 Vol=0.9 Volat=0.6 Trend=-1 | Решение: SELL +
Ситуация 18: RSI=55 MACD=0.5 Vol=0.4 Volat=0.2 Trend=0 | Решение: HOLD +
Ситуация 19: RSI=60 MACD=0.6 Vol=0.3 Volat=0.2 Trend=1 | Решение: BUY +
Ситуация 20: RSI=95 MACD=4 Vol=1 Volat=0.9 Trend=1 | Решение: SELL +
Ситуация 21: RSI=5 MACD=-6 Vol=1 Volat=1 Trend=-1 | Решение: SELL +

=== ТОЧНОСТЬ НА ОБУЧАЮЩИХ ДАННЫХ ===
Правильных ответов: 21 из 21 (100%)
```

Рисунок 2 – Вывод программы (этап обучения)

```
=== ТЕСТИРОВАНИЕ НОВЫХ РЫНОЧНЫХ СИТУАЦИЙ ===

Тест 1: Сильный восходящий тренд, высокий RSI
  Показатели: RSI=82 MACD=2.8 Volume=0.9 Volatility=0.4 Trend=1
  Рекомендация: BUY
  Уверенность: BUY=87.3229%, HOLD=17.1993%, SELL=3.8801%

Тест 2: Боковой тренд, нейтральные индикаторы
  Показатели: RSI=45 MACD=-0.5 Volume=0.5 Volatility=0.3 Trend=0
  Рекомендация: HOLD
  Уверенность: BUY=12.2944%, HOLD=81.3155%, SELL=3.05188%

Тест 3: Нисходящий тренд, низкий RSI
  Показатели: RSI=28 MACD=-2.5 Volume=0.8 Volatility=0.8 Trend=-1
  Рекомендация: SELL
  Уверенность: BUY=0.78751%, HOLD=16.8325%, SELL=81.9023%

Тест 4: Экстремальная перекупленность
  Показатели: RSI=92 MACD=3.5 Volume=1 Volatility=0.7 Trend=1
  Рекомендация: HOLD
  Уверенность: BUY=22.0515%, HOLD=28.6721%, SELL=28.4899%

Тест 5: Экстремальная перепроданность
  Показатели: RSI=12 MACD=-4.5 Volume=0.9 Volatility=1 Trend=-1
  Рекомендация: SELL
  Уверенность: BUY=0.911646%, HOLD=6.4652%, SELL=93.5427%

Тест 6: Умеренный рост, низкая волатильность
  Показатели: RSI=65 MACD=1.2 Volume=0.4 Volatility=0.2 Trend=1
  Рекомендация: BUY
  Уверенность: BUY=79.1647%, HOLD=46.6886%, SELL=0.507902%

Тест 7: Умеренное падение, средняя волатильность
  Показатели: RSI=35 MACD=-1.8 Volume=0.7 Volatility=0.6 Trend=-1
  Рекомендация: HOLD
  Уверенность: BUY=0.897948%, HOLD=50.2029%, SELL=48.5337%

Тест 8: Неопределенность на рынке
  Показатели: RSI=55 MACD=0.3 Volume=0.6 Volatility=0.5 Trend=0
  Рекомендация: HOLD
  Уверенность: BUY=3.26542%, HOLD=84.4639%, SELL=9.22355%
```

Рисунок 3 – Вывод программы (этапы проверки и тестов)

После запуска программы на экран выводится сообщение о начале обучения инвестиционного советника. Далее начинается основной этап — обучение сети. В процессе обучения через определённые промежутки времени отображается текущий шаг и значение ошибки (MSE). Это позволяет наблюдать за тем, как изменяется качество работы сети: по мере обучения ошибка должна уменьшаться, что свидетельствует о правильной настройке весов. Этот этап можно рассматривать как прогон обучающих данных с постепенной корректировкой параметров сети.

После завершения обучения программа автоматически переходит ко второму этапу —

проверке сети на обучающей выборке. Для каждого примера из набора данных выводятся значения рыночных показателей, а также решение, которое принимает нейросеть. Если результат совпадает с ожидаемым, рядом выводится отметка о правильности, в противном случае указывается, какое решение должно было быть получено. Для первых примеров дополнительно отображаются значения выходных нейронов, которые можно интерпретировать как степень уверенности сети в каждом из возможных действий. В конце этого этапа выводится общее количество правильных ответов и процент точности, что позволяет оценить, насколько хорошо сеть усвоила обучающие данные и не ограничилась ли простым запоминанием.

На третьем этапе выполняется тестирование на новых данных, которые не входили в обучающую выборку. Для каждой тестовой ситуации сначала выводится её краткое описание, затем отображаются значения входных параметров. После этого программа показывает итоговую рекомендацию нейросети — покупка, удержание или продажа. Также выводятся численные значения выходов, выраженные в процентах, которые отражают уверенность модели в каждом варианте решения. Этот этап особенно важен, так как он демонстрирует способность сети применять полученные знания к новым условиям и делать обобщённые выводы.

Код программы

```
#include <iostream>
#include <string>
#include <cmath>
#include <cstdlib>
#include <ctime>

using namespace std;

#define INPUT_NEURONS   5    // Рыночные показатели: RSI, MACD, Volume, Volatility, Trend
#define HIDDEN_NEURONS  4    // Скрытый слой
#define OUTPUT_NEURONS  3

double wih[INPUT_NEURONS + 1][HIDDEN_NEURONS];  // Веса вход - скрытый слой
double who[HIDDEN_NEURONS + 1][OUTPUT_NEURONS]; // Веса скрытый - выходной слой

double inputs[INPUT_NEURONS];
double hidden[HIDDEN_NEURONS];
double target[OUTPUT_NEURONS];
double actual[OUTPUT_NEURONS];


double erro[OUTPUT_NEURONS];
double errh[HIDDEN_NEURONS];

#define LEARN_RATE    0.15
#define RAND_WEIGHT   (((float)rand()/(float)RAND_MAX)-0.5)
#define getSRand()    ((float)rand()/(float)RAND_MAX)
#define sqr(x)        ((x)*(x))

typedef struct {
    double rsi;       // Индекс относительной силы
    double macd;      // MACD (положительный/отрицательный)
    double volume;    // Объем торгов
    double volatility; // Волатильность
    double trend;     // Тренд
    double out[OUTPUT_NEURONS];
} MARKET_DATA;

const int MAX_SAMPLES = 21;

MARKET_DATA samples[MAX_SAMPLES] = {
    // Сильный бычий рынок
    {75.0, 2.5, 0.9, 0.3, 1.0, {1.0, 0.0, 0.0}},  // Покупать
```

```
    {80.0, 3.0, 1.0, 0.4, 1.0, {1.0, 0.0, 0.0}},  // Покупать

    // Умеренный бычий рынок
    {65.0, 1.2, 0.7, 0.5, 1.0, {0.8, 0.2, 0.0}},  // Скорее покупать
    {60.0, 0.8, 0.6, 0.5, 1.0, {0.3, 0.7, 0.0}},  // Держать

    // Боковой рынок
    {50.0, 0.1, 0.5, 0.4, 0.0, {0.0, 1.0, 0.0}},  // Держать
    {45.0, -0.2, 0.4, 0.3, 0.0, {0.0, 1.0, 0.0}}, // Держать
    {55.0, 0.3, 0.6, 0.4, 0.0, {0.1, 0.9, 0.0}},  // Держать

    // Умеренный медвежий рынок
    {35.0, -1.5, 0.8, 0.7, -1.0, {0.0, 0.3, 0.7}}, // Скорее продавать
    {30.0, -2.0, 0.9, 0.8, -1.0, {0.0, 0.1, 0.9}}, // Продавать

    // Сильный медвежий рынок
    {25.0, -3.0, 1.0, 0.9, -1.0, {0.0, 0.0, 1.0}}, // Продавать
    {20.0, -4.0, 0.9, 1.0, -1.0, {0.0, 0.0, 1.0}}, // Продавать

    // Перекупленность
    {85.0, 2.0, 0.8, 0.6, 1.0, {0.0, 0.9, 0.1}},   // Возможно держать
    {90.0, 1.5, 0.7, 0.7, 0.0, {0.0, 0.8, 0.2}},   // Держать

    // Перепроданность
    {15.0, -3.5, 0.6, 0.8, -1.0, {0.2, 0.3, 0.5}}, // Продавать
    {10.0, -5.0, 0.5, 1.0, -1.0, {0.1, 0.2, 0.7}}, // Продавать

    // Высокий объем + тренд
    {70.0, 2.8, 1.0, 0.5, 1.0, {1.0, 0.0, 0.0}},  // Покупать
    {40.0, -1.8, 0.9, 0.6, -1.0, {0.0, 0.2, 0.8}}, // Продавать

    // Низкая волатильность
    {55.0, 0.5, 0.4, 0.2, 0.0, {0.2, 0.8, 0.0}},   // Держать
    {60.0, 0.6, 0.3, 0.2, 1.0, {0.6, 0.4, 0.0}},   // Скорее покупать

    // Экстремальные условия
    {95.0, 4.0, 1.0, 0.9, 1.0, {0.0, 0.3, 0.7}},   // Продавать (перекуплено)
    {5.0, -6.0, 1.0, 1.0, -1.0, {0.0, 0.1, 0.9}}   // Продавать (паника)
};

string actions[3] = { "BUY", "HOLD", "SELL" };
string indicators[5] = { "RSI", "MACD", "Volume", "Volatility", "Trend" };

void assignRandomWeights(void) {
    for (int inp = 0; inp < INPUT_NEURONS + 1; inp++) {
```

```
        for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
            wih[inp][hid] = RAND_WEIGHT;
        }
    }
    for (int hid = 0; hid < HIDDEN_NEURONS + 1; hid++) {
        for (int out = 0; out < OUTPUT_NEURONS; out++) {
            who[hid][out] = RAND_WEIGHT;
        }
    }
}

double sigmoid(double val) {
    return (1.0 / (1.0 + exp(-val)));
}

double sigmoidDerivative(double val) {
    return (val * (1.0 - val));
}

void feedForward() {
    double sum;

    for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
        sum = 0.0;
        for (int inp = 0; inp < INPUT_NEURONS; inp++) {
            sum += inputs[inp] * wih[inp][hid];
        }
        sum += wih[INPUT_NEURONS][hid];
        hidden[hid] = sigmoid(sum);
    }

    for (int out = 0; out < OUTPUT_NEURONS; out++) {
        sum = 0.0;
        for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
            sum += hidden[hid] * who[hid][out];
        }
        sum += who[HIDDEN_NEURONS][out];
        actual[out] = sigmoid(sum);
    }
}

void backPropagate(void) {
    for (int out = 0; out < OUTPUT_NEURONS; out++) {
        erro[out] = (target[out] - actual[out]) * sigmoidDerivative(actual[out]);
    }
```

```
    for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
        errh[hid] = 0.0;
        for (int out = 0; out < OUTPUT_NEURONS; out++) {
            errh[hid] += erro[out] * who[hid][out];
        }
        errh[hid] *= sigmoidDerivative(hidden[hid]);
    }

    for (int out = 0; out < OUTPUT_NEURONS; out++) {
        for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
            who[hid][out] += (LEARN_RATE * erro[out] * hidden[hid]);
        }
        who[HIDDEN_NEURONS][out] += (LEARN_RATE * erro[out]);
    }

    for (int hid = 0; hid < HIDDEN_NEURONS; hid++) {
        for (int inp = 0; inp < INPUT_NEURONS; inp++) {
            wih[inp][hid] += (LEARN_RATE * errh[hid] * inputs[inp]);
        }
        wih[INPUT_NEURONS][hid] += (LEARN_RATE * errh[hid]);
    }
}

int getAction(double* vector) {
    int sel = 0;
    double max = vector[sel];

    for (int index = 1; index < OUTPUT_NEURONS; index++) {
        if (vector[index] > max) {
            max = vector[index];
            sel = index;
        }
    }
    return sel;
}

void printMarketData(int index) {
    cout << "RSI=" << samples[index].rsi
        << " MACD=" << samples[index].macd
        << " Vol=" << samples[index].volume
        << " Volat=" << samples[index].volatility
        << " Trend=" << samples[index].trend;
}
```

```
int main() {
    setlocale(LC_ALL, "ru");
    double err;
    int correct = 0;

    srand(static_cast<unsigned int>(time(NULL)));

    cout << "=== ОБУЧЕНИЕ ИНВЕСТИЦИОННОГО СОВЕТНИКА ===\n\n";

    assignRandomWeights();

    for (int step = 0; step < 20000; step++) {
        int sample = step % MAX_SAMPLES;

        inputs[0] = samples[sample].rsi / 100.0;      // Нормализация RSI
        inputs[1] = (samples[sample].macd + 10) / 20; // Нормализация MACD (диапазон примерно -10..10)
        inputs[2] = samples[sample].volume;
        inputs[3] = samples[sample].volatility;
        inputs[4] = (samples[sample].trend + 1) / 2;  // Нормализация тренда (-1..1 -> 0..1)

        for (int i = 0; i < OUTPUT_NEURONS; i++) {
            target[i] = samples[sample].out[i];
        }

        feedForward();

        err = 0.0;
        for (int i = 0; i < OUTPUT_NEURONS; i++) {
            err += sqr(target[i] - actual[i]);
        }
        err = 0.5 * err;

        if (step % 2000 == 0) {
            cout << "Шаг " << step << " | MSE = " << err << "\n";
        }

        backPropagate();
    }

    cout << "\n=== ПРОВЕРКА НА ОБУЧАЮЩЕЙ ВЫБОРКЕ ===\n\n";

    for (int i = 0; i < MAX_SAMPLES; i++) {
        inputs[0] = samples[i].rsi / 100.0;
        inputs[1] = (samples[i].macd + 10) / 20;
        inputs[2] = samples[i].volume;
```

```
    inputs[3] = samples[i].volatility;
    inputs[4] = (samples[i].trend + 1) / 2;

    for (int j = 0; j < OUTPUT_NEURONS; j++) {
        target[j] = samples[i].out[j];
    }

    feedForward();

    int predicted = getAction(actual);
    int expected = getAction(target);

    cout << "Ситуация " << (i + 1) << ": ";
    printMarketData(i);
    cout << " | Решение: " << actions[predicted];

    if (predicted != expected) {
        cout << " (Ожидалось: " << actions[expected] << ") *";
    }
    else {
        cout << " +";
        correct++;
    }
    cout << "\n";

    if (i < 3) {
        cout << "  Вероятности: BUY=" << actual[0]
            << " HOLD=" << actual[1]
            << " SELL=" << actual[2] << "\n\n";
    }
}

cout << "\n=== ТОЧНОСТЬ НА ОБУЧАЮЩИХ ДАННЫХ ===\n";
cout << "Правильных ответов: " << correct << " из " << MAX_SAMPLES;
cout << " (" << (float)correct / MAX_SAMPLES * 100.0 << "%)\n\n";

cout << "=== ТЕСТИРОВАНИЕ НОВЫХ РЫНОЧНЫХ СИТУАЦИЙ ===\n\n";

struct {
    double rsi; double macd; double volume; double volatility; double trend;
    string description;
} tests[] = {
    {82.0, 2.8, 0.9, 0.4, 1.0, "Сильный восходящий тренд, высокий RSI"},
    {45.0, -0.5, 0.5, 0.3, 0.0, "Боковой тренд, нейтральные индикаторы"},
    {28.0, -2.5, 0.8, 0.8, -1.0, "Нисходящий тренд, низкий RSI"},
```

```
        {92.0, 3.5, 1.0, 0.7, 1.0, "Экстремальная перекупленность"},
        {12.0, -4.5, 0.9, 1.0, -1.0, "Экстремальная перепроданность"},
        {65.0, 1.2, 0.4, 0.2, 1.0, "Умеренный рост, низкая волатильность"},
        {35.0, -1.8, 0.7, 0.6, -1.0, "Умеренное падение, средняя волатильность"},
        {55.0, 0.3, 0.6, 0.5, 0.0, "Неопределенность на рынке"}
    };

    int numTests = sizeof(tests) / sizeof(tests[0]);

    for (int i = 0; i < numTests; i++) {
        inputs[0] = tests[i].rsi / 100.0;
        inputs[1] = (tests[i].macd + 10) / 20;
        inputs[2] = tests[i].volume;
        inputs[3] = tests[i].volatility;
        inputs[4] = (tests[i].trend + 1) / 2;

        feedForward();

        cout << "Тест " << (i + 1) << ": " << tests[i].description << "\n";
        cout << "  Показатели: RSI=" << tests[i].rsi
            << " MACD=" << tests[i].macd
            << " Volume=" << tests[i].volume
            << " Volatility=" << tests[i].volatility
            << " Trend=" << tests[i].trend << "\n";
        cout << "  Рекомендация: " << actions[getAction(actual)] << "\n";
        cout << "  Уверенность: BUY=" << actual[0] * 100 << "%, "
            << "HOLD=" << actual[1] * 100 << "%, "
            << "SELL=" << actual[2] * 100 << "%\n\n";
    }

    return 0;
}
```

Графики

В качестве исходной конфигурации использовалась модель с четырьмя нейронами в скрытом слое и коэффициентом обучения, равным 0.15. Для анализа процесса обучения была исследована динамика изменения ошибки по мере увеличения числа итераций. Зависимость ошибки от номера шага обучения была представлена графически: по горизонтальной оси откладывались шаги обучения, а по вертикальной — среднее значение ошибки на обучающей выборке. Полученные графики приведены на рисунках 4–5.

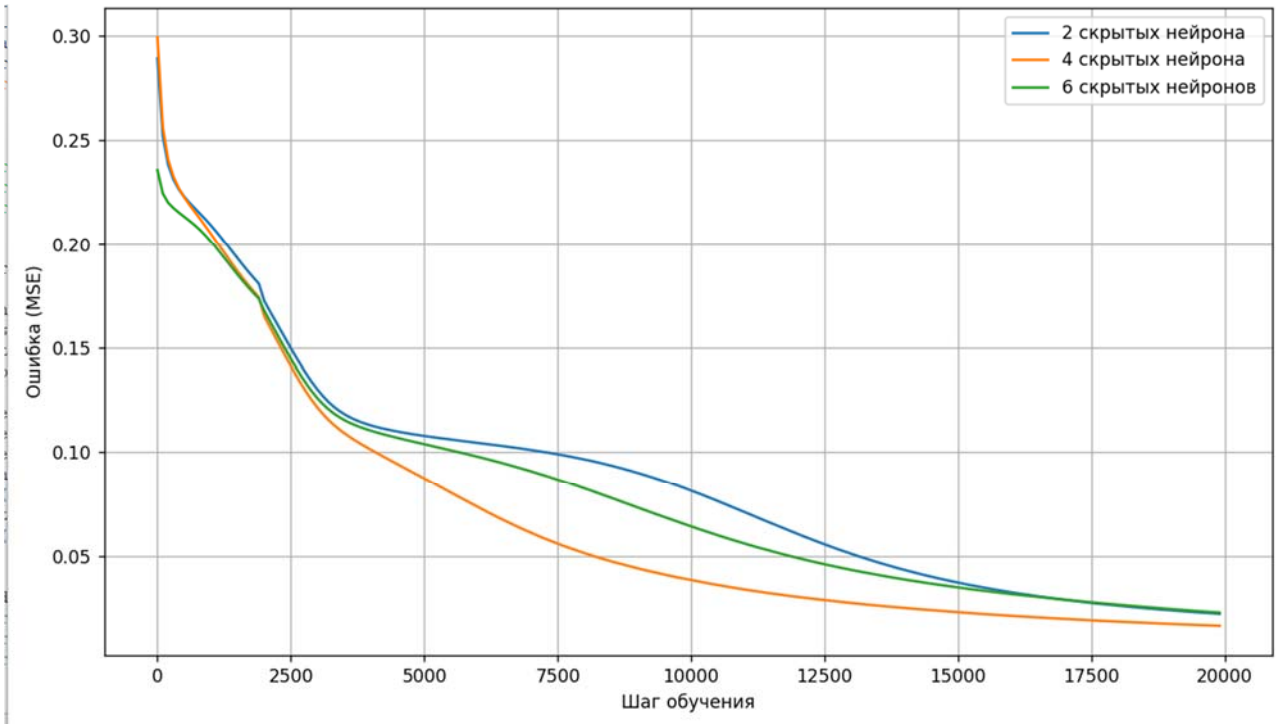

Рисунок 4 – Графики зависимостей доли ошибок от количества нейронов скрытого слоя

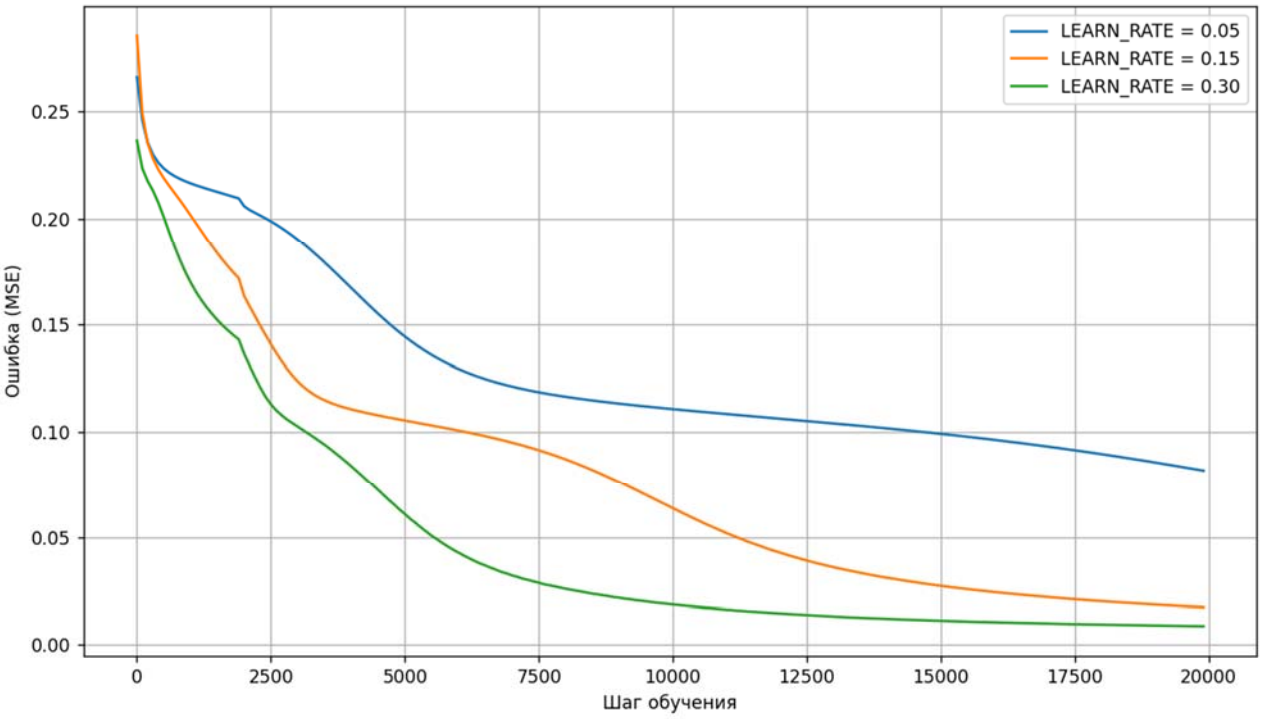

Рисунок 5 – Графики зависимостей доли ошибок от коэффициента обучения

Контрольные вопросы

- **Зачем нужен нейроконтроллер?**

Заранее прописать поведение системы для всех возможных ситуаций практически невозможно, особенно если входные данные могут сильно различаться. Поэтому используется нейроконтроллер, который после обучения на ограниченном наборе примеров способен самостоятельно принимать решения в новых условиях.

- **В каких случаях применяется алгоритм обратного распространения ошибки?**

Этот алгоритм применяется при обучении многослойных нейронных сетей, когда известны правильные ответы для обучающих примеров. Его задача — уменьшить расхождение между ожидаемым и фактическим результатом за счёт изменения весов связей. Особенно важен он для настройки скрытых слоёв, где напрямую задать корректные значения невозможно.

- **Какая функция активации используется для алгоритма обратного распространения ошибки?**

В данной программе используется сигмоидальная функция активации. Она удобна тем, что ограничивает значения в диапазоне от 0 до 1 и имеет простую производную, что важно для обучения. В целом могут применяться и другие функции, главное условие — они должны быть дифференцируемыми, чтобы можно было корректно вычислять градиенты: Функция гиперболического тангенса, Функция арктангенса, гудерманская функция, функция ошибки, обобщенная логистическая функция, функция Smoothstep, некоторые алгебраические функции и т.д.

- **Как инициализируются веса при использовании алгоритма обратного распространения ошибки?**

Перед началом обучения все веса задаются случайными значениями из небольшого диапазона. Это необходимо, чтобы нейроны не работали одинаково и сеть могла начать обучение без смещения в какую-либо сторону.

- **Как вычисляется ошибка слоя?**

Для выходного слоя ошибка определяется как разность между ожидаемым и полученным значением, умноженная на производную функции активации. Для скрытого слоя ошибка рассчитывается через ошибки выходного слоя: суммируются их вклады с учётом весов связей, после чего результат также умножается на производную

функции активации.

- **Как корректируются веса?**

Изменение весов происходит пропорционально величине ошибки, значению сигнала и коэффициенту обучения. К каждому весу прибавляется произведение скорости обучения, ошибки соответствующего нейрона и входного значения или активации. За счёт этого сеть постепенно подстраивается и уменьшает общую ошибку

- **Как формируется ответ на эффекторном слое?**

После прохождения сигнала через сеть формируются значения на выходных нейронах. Каждое из них соответствует одному из возможных действий. Итоговое решение определяется по наибольшему значению — именно оно считается выбранным действием.

- **Как контролируется обучение нейроконтроллера?**

Контроль осуществляется по нескольким показателям. Во-первых, отслеживается изменение ошибки в процессе обучения. Во-вторых, проверяется точность на обучающей выборке. И, наконец, проводится тестирование на новых данных, что позволяет оценить, насколько хорошо сеть умеет обобщать и применять полученные знания в незнакомых ситуациях.